LoRA: Low-Rank Adaptation of Large Language Models

A reduced-scale reproduction and rank ablation (DL-051)

Manya Dixit ▪ 24CS2026 ▪ IDD

October 2026

1. Objective and the problem with full fine-tuning

Objective: implement LoRA *from scratch* (no PEFT), plug it into pretrained DistilBERT for SST-2 sentiment, compare against full fine-tuning, ablate the rank $r \in \{2, 4, 8\}$.

Full fine-tuning is expensive because:

- every one of the 67M weights (or 175B for GPT-3) is updated per task;
- Adam stores *two extra fp32 buffers per parameter* during training;
- each task needs its own full model copy to store and serve.
- GPT-3 175B: 1.2 TB training memory; 350 GB task checkpoint (Hu et al., ICLR 2022).

2. LoRA: intuition and the key equation

Idea: the fine-tuning *change* ΔW has low intrinsic rank, so learn it as a product of two thin matrices:

$$h = W_0 x + \frac{\alpha}{r} BAx, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

- W_0 **frozen**; only A, B (+ classifier head) train.
- B init $= 0 \Rightarrow \Delta W = 0$ at step 0: training starts *exactly* at the pretrained function.
- Trainable cost per layer: $dk \to r(d + k)$ (linear in r).
- $W_0 + BA$ merges back at deployment $\Rightarrow$ no inference latency.
- $\alpha = 2r$ here, so the scale $\alpha / r = 2$ is constant across ranks.

3. Custom implementation

`src/lora.py` — plain PyTorch, no `peft` (enforced by a test):

```
class LoRALinear(nn.Module):
    def forward(self, x):
        base = self.base_layer(x)          # frozen W0
        x    = self.lora_dropout(x)
        return base + self.scaling * F.linear(
                   F.linear(x, self.lora_A), self.lora_B)
```

- Injection wraps 6 layers $\times$ {query, value} $=$ 12 linears (paper adapts W_q, W_v).
- 16 unit tests + 11 smoke checks: shapes, B=0 identity, merged-weight equivalence $W' = W_0 + \frac{\alpha}{r} BA$, gradient flow, **frozen backbone bit-identical after optimizer steps**.

4. Experimental setup

- **Model:** `distilbert-base-uncased` (67.0M params), fresh pretrained checkpoint per run.
- **Data:** GLUE SST-2 (67,349 train / 872 val; labels 0=neg, 1=pos); fixed seeded subset of 8,000 training examples shared by ALL runs (indices persisted for audit).
- **Protocol:** seq len 128, batch 16, 2 epochs, seed 42, fp32, AdamW; LR 2×10^{-5} (full) vs 10^{-3} (LoRA); eval on validation after every epoch, best checkpoint kept.
- **Device:** measured on mps; peak memory from PyTorch allocator stats (sampled per batch on MPS).

5. Results: full fine-tuning vs LoRA

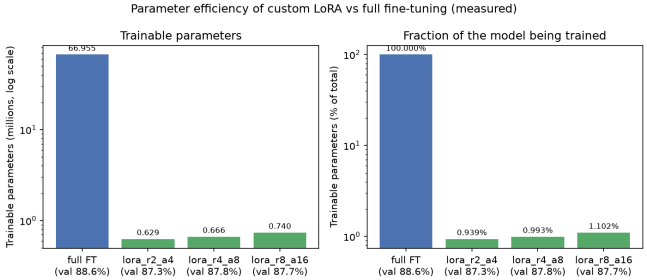
Method	Trainable params	Train %	Val acc	Time (min)
Full FT	66,955,010	100.000%%	88.65%	10.6
LoRA r=2	628,994	0.939%%	87.27%	5.9
LoRA r=4	665,858	0.993%%	87.84%	6.0
LoRA r=8	739,586	1.102%%	87.73%	6.0

Values measured in this project (see `results/results.csv`); paper's RoBERTa-large SST-2: 96.4 (full) vs 96.2±0.5 (LoRA r=4) — different model/data, so compare the *pattern*, not the numbers. **Cross-device confirmation** (Google Colab T4, identical protocol): full 89.11% vs LoRA r=4 88.76% at 0.99% trainable; peak mem 1665 MiB vs 744 MiB (2.2×).

6. Rank ablation

- Adapter parameters scale *exactly linearly* with r: $6 \times 2 \times r \times (768{+}768) = 18{,}432\,r$ params (36,864 / 73,728 / 147,456 for $r{=}2/4/8$) – verified by counting the live model, not by the formula.
- Accuracy: 87.27% ($r{=}2$) $\rightarrow$ 87.84% ($r{=}4$) $\rightarrow$ 87.73% ($r{=}8$).
- Matches the paper's GPT-3 finding: adapting $W_q{+}W_v$, accuracy is essentially flat from $r{=}1$ to $r{=}64$ (WikiSQL 73.4–73.8) — small ranks already suffice.

7. Memory and parameter efficiency



- Trainable parameters: 66,955,010 → 665,858 ($\approx \sim 100\times$ fewer for r=4).
- Optimizer state and gradients exist *only* for trainable params, so LoRA's training-state memory shrinks with it.
- But the frozen backbone must still be stored + forwarded: peak memory does not drop by $100\times$ (measured 1027 MiB (full FT) vs 265 MiB (LoRA), i.e. $\approx 4\times$).

8. Limitations and failure cases

- Single seed, 8,000-example subset: ± 0.5pt differences between ranks are within noise — we do not over-interpret them.
- Validation split used for evaluation AND best-epoch selection (GLUE test labels are withheld).
- Local device is mps, not the target Colab T4: absolute memory/time figures are device-specific (notebook reproduces on CUDA).
- DistilBERT + SST-2 is an *adaptation* of the paper's principle — the paper reports RoBERTa/GPT-2/GPT-3, never DistilBERT.
- No secondary ablation (placement / α rule) – main experiments took priority, as the assignment allows.

9. Conclusion

- From-scratch LoRA ($h = W_0x + \frac{\alpha}{r}BAx$, B=0 init, W_0 frozen) integrated into DistilBERT and unit-tested.
- At 8,000 SST-2 examples, LoRA matches full fine-tuning within $\sim$**1 point** of validation accuracy while training $\sim$**100**$\times$ **fewer** parameters (r=4).
- Rank controls capacity linearly; accuracy saturates at small r — consistent with the paper's findings at 175B scale.
- Adapters merge into W_0: no extra inference cost, tiny per-task checkpoints.

Ref: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022 (arXiv:2106.09685); Sanh et al. 2019 (DistilBERT); Socher et al. 2013 / Wang et al. 2018 (SST-2 / GLUE).

10. Q&A preparation

Be ready to derive and explain:

- Why $r(d+k)$ parameters? Why is BA rank-$\leq r$?
- Why B=0 (and why that makes $\partial \mathcal{L}/\partial A = 0$ at step 0)?
- Why adapt W_q and W_v specifically?
- Where do the optimizer-state savings come from, and what does LoRA *not* save?

Full list with answers: `presentation/viva_prep.md`.